

Initialization

Converting g_2-1 to $g_1(\tau)$

The function takes $\sqrt{\frac{g_2-1}{\text{Max}[g_2-1]}}$ and fits with $\text{Exp}(-a t)$, then all the points below the threshold2 are clipped to 0, and between threshold1 and threshold2 are removed

```
In[3]:= ClearAll[clipG2toG1];

clipG2toG1[g2data_, glthCut_ : 10*10^-3, glthZero_ : 1*10^-4, nkeep_ : 13] :=
Module[{g1, fit, tCut, tZero, glkeep, tail, glzeros, norm}, (*g1 from g2
  (same normalization as your original code)*) norm = Max[g2data[[All, 2]]];
  g1 = xySc[g2data, Identity, Sqrt[#/norm] &];
  (*single exponential fit*)
  fit = FindFit[g1, Exp[-a t], a, t];
  (*two cutoff times from the fit*)
  tCut = (-Log[glthCut] / a) /. fit; (*keep data up to here*)
  tZero = (-Log[glthZero] / a) /. fit;
  (*start zero-padding here*)
  (*keep only real data up to tCut*) glkeep = Select[g1, #[[1]] ≤ tCut &];
  (*take first nkeep points at/after tZero,
  keep their original times*) tail = Select[g1, #[[1]] ≥ tZero &];
  glzeros = ({#[[1]], 0} &) /@ Take[tail, UpTo[nkeep]];
  (*return: data region+explicit zero-padding,
  with the gap deleted*) Join[glkeep, glzeros]]
```

MEM solvers

Unconstrained MEM interior point IPOPT on simplex

The module for MEM deconvolution of exponential decay functions. It takes a list in format: {time, $g^{(2)}-1$ }, step in γ -space on log scale, and regularization parameter λ . It returns generated γ -list and recovered distribution of a_i , residuals $(\hat{g}_1 - g_1)^2$ of fit and entropy $S = -a_i \log(a_i)$. It uses NMinimize and IPOPT method.

```
In[5]:= ClearAll[dlsMEMu];
```

```
dlsMEMu[dlsData_List, step_ : 0.1, λ_ : 5 × 10-5] := Module[
  {tauList, g1Noisy, gammaList, n, aVars, kernel, gammaTauMatrix, gammaCutoff = 50,
   solution, aRecovered, modelG1Fit, residualVec, entropyVec, eps = 10-12, a},
  (*1. Extract data*)
  tauList = dlsData[[All, 1]];
  g1Noisy = dlsData[[All, 2]];
  (*2. Gamma grid*)
  gammaList = Exp[Range[Log[1 / Max[tauList]], Log[1 / Min[tauList]], step]];
  n = Length[gammaList];
  (*3. Distribution amplitudes a_i*)
  aVars = Array[a, n];
  (*4. Kernel*)
  gammaTauMatrix = Clip[Outer[Times, tauList, gammaList], {0, gammaCutoff}];
  kernel = Exp[-gammaTauMatrix];
  (*5. Objective with simplex constraint*)
  solution = NMinimize[{Module[{modelG1, kappa, entropy}, modelG1 = kernel.aVars;
    kappa = Total[(modelG1 - g1Noisy) ^ 2];
    entropy = -Total[aVars Log[aVars + eps]];
    kappa - λ entropy], Thread[aVars ≥ eps] && Total[aVars] == 1},
    aVars, Method → "ILOPT", MaxIterations → 300];
  (*6. Recover solution*)
  aRecovered = aVars /. solution[[2]];
  modelG1Fit = kernel.aRecovered;
  residualVec = (modelG1Fit - g1Noisy) ^ 2;
  entropyVec = -aRecovered Log[aRecovered + eps];
  {gammaList, aRecovered, residualVec, entropyVec}]
```

Constrained MEM, Interior Point on simplex

The module for MEM deconvolution of exponential decay functions. It takes a list in format: {time, $g^{(2)}-1$ }, step in γ -space on log scale, and an instrument constant C. It returns generated γ -list and recovered distribution of a_i , residuals $(\hat{g}_i - g_i)^2$ of fit and entropy $S = -a_i \log(a_i)$. It uses NMaximize and IPOPT method.

```
In[7]:= ClearAll[dlsMEMc];
```

```
dlsMEMc[dlsData_List, step_ : 0.1, const_ :  $6. \times 10^{-4}$ ] := Module[
  {tauList, g1Noisy, gammaList, n, aVars, kernel, gammaTauMatrix, gammaCutoff = 50,
   solution, aRecovered, modelG1Fit, residualVec, entropyVec, eps =  $10^{-12}$ , a},
  (*1. Extract data*)tauList = dlsData[[All, 1]];
  g1Noisy = dlsData[[All, 2]];
  (*2. Gamma grid*)
  gammaList = Exp[Range[Log[1 / Max[tauList]], Log[1 / Min[tauList]], step]];
  n = Length[gammaList];
  (*3. Distribution amplitudes a_i*)
  aVars = Array[a, n];
  (*4. Kernel*)
  gammaTauMatrix = Clip[Outer[Times, tauList, gammaList], {0, gammaCutoff}];
  kernel = Exp[-gammaTauMatrix];
  (*5. Constrained MEM*)
  solution =
    NMaximize[{-Total[aVars Log[(aVars + eps)]]], Thread[aVars  $\geq 10^{-12}$ ] &&
      Total[aVars] == 1 && Total[(kernel.aVars - g1Noisy) ^ 2]  $\leq$  const},
    aVars, Method  $\rightarrow$  {"IPOPT", "Tolerance"  $\rightarrow 10^{-5}$ , "MaxIterations"  $\rightarrow 200$ ]];
  (*6. Recover solution*)
  aRecovered = aVars /. solution[[2]];
  modelG1Fit = kernel.aRecovered;
  residualVec = (modelG1Fit - g1Noisy) ^ 2;
  entropyVec = -aRecovered Log[aRecovered + eps];
  {gammaList, aRecovered, residualVec, entropyVec}]
```

Diagnostics

The function that computes the **memPeakHalfInterp** extracts the dominant peak position and the left half-width from a MEM spectrum {gammaList, aRecovered, residualVec, entropyVec}. The distribution is converted to time domain ($\tau = 1/\gamma$), interpolated, and the left half-maximum is solved numerically.

If a single dataset is supplied, the function returns {tauHalf, tauPeak, CV}.

If a list of datasets is supplied, the peak of the first dataset is used as a reference and CV values are computed relative to that peak for all datasets.

```
In[9]:= ClearAll[memPeakHalfInterp]
```

```
memPeakHalfInterp[mem_, level_ : 0.5, thrFrac_ : 0.01] := Module[
  {compute, tauPeakRef}, (*compute peak and left width for one MEM dataset*)
  compute[data_] := Module[{glist, a, tau, d, peakIdx, tauPeak,
    aPeak, left, thr, pairs, i, tauHalf}, glist = data[[1]];
    a = Clip[data[[2]], {0, Infinity}];
    tau = 1 / glist;
    d = SortBy[Transpose[{tau, a}], First];
    (*peak position*) peakIdx = First@Ordering[d[[All, 2]], -1];
    tauPeak = d[[peakIdx, 1]];
    aPeak = d[[peakIdx, 2]];
    (*keep left side of peak*) left = d[[1 ;; peakIdx]];
    (*remove entropy tail*) thr = thrFrac aPeak;
    left = Select[left, #[[2]] > thr &];
    (*find pair bracketing the chosen level*)
    pairs = Partition[left[[All, 2]], 2, 1];
    i = First@FirstPosition[pairs, {y1_, y2_} /; y1 ≤ level aPeak ≤ y2];
    (*linear interpolation for half width*)
    tauHalf = left[[i, 1]] + (level aPeak - left[[i, 2]]) *
      (left[[i + 1, 1]] - left[[i, 1]]) / (left[[i + 1, 2]] - left[[i, 2]]);
    {tauHalf, tauPeak}];
  (*-----single MEM dataset-----*)
  If[Length[mem] == 4 && VectorQ[mem[[1]]] && VectorQ[mem[[2]]],
    Module[{tauHalf, tauPeak, cv}, {tauHalf, tauPeak} = compute[mem];
      cv = Abs[tauPeak - tauHalf] / (1.177 tauPeak);
      (*return:{halfwidth position, peak position, CV}*)
      {tauHalf, tauPeak, cv}],
    (*-----list of MEM datasets-----*)
    Module[{results, tauHalfList, tauPeakList, cv},
      (*reference peak from first dataset*) tauPeakRef = compute[mem[[1]]][[2]];
      (*compute peak and halfwidth for all datasets*) results = compute /@ mem;
      tauHalfList = results[[All, 1]];
      tauPeakList = results[[All, 2]];
      (*CV relative to reference peak*)
      cv = Abs[tauHalfList - tauPeakRef] / (1.177 tauPeakRef);
      (*return:{halfwidths, peaks, CV values}*) {tauHalfList, tauPeakList, cv}]]]
```

```
In[11]:= ClearAll[memPeakHalfInterpSelf]
```

```
memPeakHalfInterpSelf[mem_, level_ : 0.5, thrFrac_ : 0.01] :=
Module[{compute}, (*compute peak and left width for one MEM dataset*)
  compute[data_] := Module[{glist, a, tau, d, peakIdx, tauPeak,
    aPeak, left, thr, pairs, i, tauHalf}, glist = data[[1]];
    a = Clip[data[[2]], {0, Infinity}];
    tau = 1 / glist;
    d = SortBy[Transpose[{tau, a}], First];
    (*peak*) peakIdx = First@Ordering[d[[All, 2]], -1];
    tauPeak = d[[peakIdx, 1]];
    aPeak = d[[peakIdx, 2]];
    (*keep left side*) left = d[[1 ;; peakIdx]];
    (*remove entropy tail*) thr = thrFrac aPeak;
    left = Select[left, #[[2]] > thr &];
    (*find pair bracketing level*) pairs = Partition[left[[All, 2]], 2, 1];
    i = First@FirstPosition[pairs, {y1_, y2_} /; y1 ≤ level aPeak ≤ y2];
    (*linear interpolation*) tauHalf = left[[i, 1]] + (level aPeak - left[[i, 2]]) *
      (left[[i + 1, 1]] - left[[i, 1]]) / (left[[i + 1, 2]] - left[[i, 2]]);
    {tauHalf, tauPeak}];
  (*single MEM dataset*) If[Length[mem] == 4 && VectorQ[mem[[1]]] && VectorQ[mem[[2]]],
    Module[{tauHalf, tauPeak, cv}, {tauHalf, tauPeak} = compute[mem];
      cv = Abs[tauPeak - tauHalf] / (1.177 tauPeak);
      {tauHalf, tauPeak, cv}], (*list of MEM datasets*)
    Module[{results, cv}, results = compute /@ mem;
      (*CV relative to each peak*)
      cv = Abs[results[[All, 2]] - results[[All, 1]]] / (1.177 results[[All, 2]]);
      {results[[All, 1]], results[[All, 2]], cv}]]]
```

Handling the data

The MEM script grid in on the log scale, so returned values are attached to the point in gamma space. They are intensity weighted. In order to transform to density weight, the intensity at $a(\gamma_i)$ should be divided by the length of the interval, i.e. $\gamma_i - \gamma_{i+1}$, then the probability function will be recovered. Since it is a log spaced, the density for the longer times will be smaller. The following function converts intensity returned from dlsMEMc or dlsMEMu to probability. In the function code, the x is time, i.e. γ and a is a Recovered.

```
In[13]:= ClearAll[pdfLinear]
```

```
pdfLinear[data_List] := Module[{x, a, n, dx}, {x, a} = Transpose[data];
  n = Length[x];
  dx = Abs@Join[{x[[2]] - x[[1]]}, {x[[3 ;;]] - x[[1 ;; -3]]} / 2, {x[[-1]] - x[[-2]]}];
  Transpose[{x, a / dx}]
```

The function to determine the knee position on the L-curve of logCV vs LogC. Returns the index of the knee point, and corresponding constant C, and CV value.

```
In[15]:= ClearAll[kneeLCurve]
```

```
kneeLCurve[Cgrid_, CV_] := Module[{x, y, p1, p2, d, pos},
  (*log-log coordinates*)
  x = Log[Cgrid];
  y = Log[CV];
  (*endpoints*)
  p1 = {x[[1]], y[[1]]};
  p2 = {x[[-1]], y[[-1]]};
  (*perpendicular distances to line*)
  d = Abs[(p2[[2]] - p1[[2]]) x - (p2[[1]] - p1[[1]]) y + p2[[1]] x p1[[2]] - p2[[2]] x p1[[1]]) /
    Sqrt[(p2[[2]] - p1[[2]])^2 + (p2[[1]] - p1[[1]])^2];
  (*knee position*)
  pos = Ordering[d, -1][[1]];
  {pos, Cgrid[[pos]], CV[[pos]]}
```

The function to compute the C-sweep diagnostics from an *.h5 file

```
In[17]:= ClearAll[memScanAnalysis]
```

```
memScanAnalysis[file_] := Module[{gamma, aRec, res, ent, memout, cs, dia, kne},
  gamma = Import[file, "gammaList"];
  aRec = Import[file, "aRecovered"];
  res = Import[file, "residualVec"];
  ent = Import[file, "entropyVec"];
  memout = Table[{gamma, aRec[[i]], res[[i]], ent[[i]]}, {i, Length[aRec]}];
  cs = Import[file, "C_constants"];
  dia = memPeakHalfInterp[memout];
  kne = kneeLCurve[cs, dia[[3]]];
  {cs, dia, kne}
```

The hdf5 files for data storage and reuse. Consists of ISO8601 time stamp and export function.

Takes the λ value and the output of dlsMEMlog in {gammaList, gRecovered, residualVal, entropyVal} format.

```
In[19]:= timeStamp[] := DateString[{"ISODateTime"}, TimeZone → 0]
```

Producing hdf5 file association containing data and metadata

```
In[20]:= makeH5Data[λ_, memOut_] :=
  Module[{gamma, f, residual, entropy}, {gamma, f, residual, entropy} = memOut;
  <|"data" → <|"lambda" → λ, "gammaList" → gamma,
    "gRecovered" → f, "residualVal" → residual, "entropyVal" → entropy|>,
  "meta" → <|"timestamp" → timeStamp[], "log10lambda" → Log10[λ], "solver" →
    "Wolfram 14.3 NMinimize", "method" → "Differential Evolution"|>|>]
```

Function to convert from machine real number of λ to text string used in h5 name generation

Time to hydrodynamic radius

The function converting the radius to time in seconds using Stokes-Einstein and Malvern geometry at RT in water.

```
In[21]:= RadiusNMToTau[Rnm_, lambda_ : 633 * 10^-9, theta_ : 173 Degree, n_ : 1.33,
  T_ : 298, eta_ : 0.00089] := Module[{kB, q, R}, kB = 1.380649 * 10^-23;
  q = (4 Pi n / lambda) Sin[theta / 2];
  R = Rnm * 10^-9;
  (6 Pi eta R) / (kB T q^2)]
```

The inverse to above function that converts time, to radius using Stokes-Einstein formalism to radius in nm.

```
In[22]:= TauToRadiusNM[tau_, lambda_ : 633 * 10^-9, (*laser wavelength,m*)
  theta_ : 173 Degree, (*scattering angle*)n_ : 1.33, (*refractive index of water*)
  T_ : 298, (*temperature,K*)eta_ : 0.00089 (*viscosity of water,Pa s*)] :=
Module[{kB, q, R}, kB = 1.380649 * 10^-23;
  (*Boltzmann constant*)q = (4 Pi n / lambda) Sin[theta / 2];
  R = (kB T q^2 tau) / (6 Pi eta); (*radius in meters*)
  Round[R * 10^9, 0.1] (*convert to nm*)]
```